Optimising PySpark Code for Performance

In this train, we'll explore how to adjust and optimize PySpark code to prevent future errors and improve application performance. We'll discuss best practices for writing efficient PySpark code.

Learning Objectives

By the end of this train, you should be able to:

- Prevent future errors and improve application performance

1. Understanding Spark Execution

Key Concepts:

- **Lazy Evaluation**: Spark doesn't execute transformations until an action is called
- **DAG (Directed Acyclic Graph)**: Spark builds an execution plan before running
- **Catalyst Optimizer**: Spark's query optimization engine

Optimization Implications:

Structure your code to allow maximum optimization by Catalyst

Be mindful of actions that trigger execution

2. Partitioning Strategies

Best Practices:

```
# Optimal partition sizing (aim for 128MB-200MB per partition)

df = df.repartition(200)  # Explicit repartitioning
df = df.coalesce(100)     # Reduce partitions without shuffle
# Partition by key for join operations

df = df.repartition("join_key")
```

Considerations:

Too many partitions → excessive overhead

Too few partitions → underutilized resources and OOM risks

Ideal partition count: total data size / block size (default 128MB)

3. Caching and Persistence

When to Cache:

Data reused multiple times

Iterative algorithms

Frequently accessed reference data

```
from pyspark import StorageLevel

df.persist(StorageLevel.MEMORY_AND_DISK)  # Default
df.persist(StorageLevel.MEMORY_ONLY)      # For small datasets
df.persist(StorageLevel.DISK_ONLY)        # For very large datasets
```

4. Efficient Transformations

Preferred Operations:

```
# Use select() early to limit columns


df.select("col1", "col2").filter(...)

# Use column expressions instead of UDFs when possible
from pyspark.sql import functions as F
df.withColumn("new_col", F.col("a") + F.col("b"))

# For complex UDFs, use pandas UDFs (Vectorized UDFs)
@pandas_udf("double")
def pandas_plus(a: pd.Series, b: pd.Series) -> pd.Series:
    return a + b
```

> **Avoid**
>
> - Collecting large datasets to driver
> - Multiple shuffles (groupBy, repartition, join)
> - UDFs when native functions exist

Conclusion

This lesson has equipped you with essential techniques to optimize PySpark code for better performance and reliability. By understanding partitioning strategies, efficient transformations, join optimizations, caching, and Spark configuration tuning, you can significantly reduce execution time and prevent common errors. Remember that effective optimization requires balancing resource usage, minimizing shuffles, leveraging Spark's built-in optimizations, and continuously monitoring performance through tools like the Spark UI. Applying these best practices will help you build scalable, efficient PySpark applications that handle large datasets effectively while avoiding costly bottlenecks.

▼Key Takeaways

- **Takeaway 1:** Partition wisely, balance partition size and count
- **Takeaway 2:** Minimize shuffles, the most expensive operation
- **Takeaway 3:** Cache strategically, but don't over-cache
- **Takeaway 4:** To use native functions, avoid UDFs when possible
- **Takeaway 5:** Tune configurations, match to your workload